

LAB 08 — Support Vector Machines

Kernels, Margins, and Model Selection on Real Customer-Churn Data

Course	Machine Learning — BS Computer Science, 5th Semester
Duration	3 hours (single session)
Dataset	Telco Customer Churn (Kaggle) — same dataset used in Labs 2, 3, 4 and 6
Prior Labs Used	Lab 2 (cleaning/EDA), Lab 3 (Decision Tree), Lab 4 (Random Forest), Lab 6 (Logistic Regression, KNN)
Deliverable	One completed Colab notebook (see Submission Requirements)

Learning Objectives

- Explain, in your own words, why feature scaling is not optional for SVM (unlike Decision Trees/Random Forest).
- Train and compare SVM classifiers with linear, RBF and polynomial kernels, and justify a kernel choice using evidence.
- Visualize how the margin and decision boundary change with the C hyperparameter, and connect this to overfitting/underfitting.
- Run a systematic C/gamma search (GridSearchCV) for an RBF-kernel SVM and interpret the resulting score surface.
- Handle class imbalance in a churn-prediction context using class_weight and evaluate the precision/recall trade-off it creates.
- Position the tuned SVM against Decision Tree, Random Forest, Logistic Regression and KNN results from earlier labs, and defend a final model choice.

Before You Start

- You already studied SVM theory in class: the margin, support vectors, the kernel trick, and the role of C and gamma. This lab does NOT re-teach that theory.
- This lab is intentionally NOT a 'load data → SVC().fit() → done' exercise. A single default SVC() call will get you a mediocre, unexamined model — most of the marks in this lab come from the experimentation, visualization and justification tasks, not from the fact that a model trained.
- You already know Python, Pandas, Matplotlib, train_test_split, encoding and evaluation metrics from Labs 2-6. None of that is re-taught here either.
- Every task below gives you an objective and a question to answer — not a code recipe. Hints tell you which scikit-learn tool is relevant; you decide how to use it.

Part 0 — Recap and Setup (10 minutes)

You are continuing the same churn-prediction project from Labs 2, 3, 4 and 6. Reload the Telco Customer Churn dataset, and reapply the SAME cleaning and encoding pipeline you already built: convert TotalCharges to numeric, handle its missing values, drop customerID, map Churn to 0/1, one-hot encode the categorical columns, and split into train/test with stratify=y.

Carry Forward, Do Not Rebuild

- If you saved a cleaned CSV from Lab 2/3, you may load it directly instead of re-cleaning from scratch.
- Use the exact same `random_state` and `test_size` you used in Labs 3/4/6 so your results stay comparable across labs.

Part 1 — Why Scaling Matters for SVM (20 minutes)

Decision Trees and Random Forest do not care about feature scale. SVM absolutely does — its margin is a geometric, distance-based concept, and distances are dominated by whichever feature happens to have the largest numeric range (look at `MonthlyCharges/TotalCharges` vs. the 0/1 dummy columns).

Task 1.1 — Objective

Demonstrate, with evidence from your own run, what happens to an SVM when features are not scaled.

What to do

1. Train a linear-kernel SVM (`SVC(kernel='linear')`) directly on your UN-scaled encoded features.
2. Train a second linear-kernel SVM on the SAME split after applying `StandardScaler` (fit on train only, transform on train and test — the same discipline rule from Labs 4-6).
3. Compare accuracy, F1-score, training time, and any convergence warnings between the two runs.

Questions to answer in Markdown

- (a) What differences did you observe between the unscaled and scaled models (accuracy, warnings, training time)?
- (b) Why does an unscaled linear SVM behave this way, in terms of how the margin/decision boundary is computed?
- (c) From this point onward in the lab, which version will you use for every subsequent SVM, and why?

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# unscaled
svm_raw = SVC(kernel='linear', random_state=42)
svm_raw.fit(X_train, y_train)

# scaled (fit_transform on TRAIN, transform ONLY on TEST)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
```

Part 2 — Kernel Comparison (35 minutes)

SVM's real flexibility comes from the kernel. A default kernel choice is a guess, not an engineering decision. Your job in this task is to make that decision with evidence.

Task 2.1 — Objective

Train and fairly compare SVM classifiers using the linear, RBF and polynomial kernels on your SCALED features.

What to do

1. Train three SVC models — `kernel='linear'`, `kernel='rbf'`, `kernel='poly'` — with the same `random_state`, on the same scaled train split.
2. For each model, record: Accuracy, Precision, Recall, F1-score, and the number of support vectors (`model.support_.shape[0]`).
3. Assemble a single comparison table (a DataFrame is fine) with one row per kernel.
4. Plot the confusion matrix for whichever kernel performed best on F1-score.

Syntax / Function	Description
<code>SVC(kernel=...)</code>	'linear', 'rbf', or 'poly' — the kernel function used to compute similarity between points.
<code>model.support_</code>	Array of indices of the training points selected as support vectors.
<code>model.n_support_</code>	Number of support vectors per class.

Questions to answer

- (a) Which kernel produced the best F1-score on the test set? Does this surprise you given the theory covered in class?
- (b) Which kernel used the most support vectors, and what does a high support-vector count suggest about the complexity of the decision boundary it learned?
- (c) Based on this evidence, which kernel would you carry forward for hyperparameter tuning in Part 4? Justify with numbers, not intuition.

Part 3 — Visualizing the Margin: What Does C Actually Do? (35 minutes)

C and gamma are not just numbers you grid-search blindly — they visibly reshape the decision boundary. Since your real feature space has 25-30 dimensions, you cannot plot it directly. You will build a 2-D PROJECTION so the boundary becomes visible, understanding clearly that this is a simplification for intuition, not the actual high-dimensional model.

Task 3.1 — Objective

Reduce your scaled features to 2 dimensions and visualize how an RBF-kernel SVM's decision boundary changes as C increases.

What to do

1. Use `PCA(n_components=2)` on your SCALED training features to obtain a 2-D projection. Report the total explained variance ratio of the two components — this tells you how much information the plot is allowed to claim to show.
2. Train three RBF-kernel SVMs on this 2-D projection, using a small, medium and large value of C (choose your own three values spanning at least two orders of magnitude, e.g. 0.01, 1, 100).
3. For each of the three C values, plot the 2-D decision boundary (use a mesh grid + `contourf` or `contour`, colored by predicted class) with the actual training points overlaid.
4. On each plot, report the number of support vectors used.

Hint — Building the Mesh Grid

- Create a fine grid of (x, y) points spanning slightly beyond your projected data's min/max range using `np.meshgrid`.
- Call `model.predict()` on every grid point (flatten the grid first, then reshape the predictions back to the grid shape).
- `plt.contourf(xx, yy, Z, alpha=0.3)` draws the shaded decision regions; `plt.scatter(...)` overlays the real points colored by true label.

Questions to answer

- How does the shape/smoothness of the decision boundary change as C increases from your smallest to largest value?
- How does the number of support vectors change with C , and does this match what you expected from the theory (C controls the margin-violation penalty)?
- Which of your three C values looks like it is overfitting on this 2-D view, and which looks like it is underfitting? Explain the visual evidence.
- Why is this 2-D plot only an APPROXIMATE illustration of what the real (high-dimensional) SVM is doing? What could it be misleading you about?

Part 4 — Systematic Hyperparameter Search (30 minutes)

Now tune the model properly, on the FULL feature set (not the 2-D projection), using cross-validation rather than eyeballing plots.

Task 4.1 — Objective

Run `GridSearchCV` over a C x gamma grid for an RBF-kernel SVM and select the best configuration using cross-validated F1-score.

What to do

- Define a parameter grid with at least 3 values of C and at least 3 values of gamma (choose sensible ranges spanning multiple orders of magnitude).
- Run `GridSearchCV(SVC(kernel='rbf'), param_grid, cv=5, scoring='f1')` on your SCALED training data.
- Extract the full results (`gs.cv_results_`) and reshape the mean test scores into a C x gamma grid.
- Visualize this grid as a heatmap (matplotlib `imshow` or a Pandas-pivoted DataFrame is fine) with C and gamma labeled on the axes.
- Refit the best estimator and report its performance (accuracy, precision, recall, F1, confusion matrix) on the held-out TEST set.

Syntax / Function	Description
<code>GridSearchCV(estimator, param_grid, cv, scoring)</code>	Exhaustively tries every combination in <code>param_grid</code> using k-fold cross-validation.
<code>gs.best_params_</code>	The hyperparameter combination with the best mean cross-validated score.
<code>gs.cv_results_['mean_test_score']</code>	Array of mean CV scores for every combination tried, in the same order as the grid was expanded.

Questions to answer

- (a) What were the best C and gamma values found, and what was the corresponding cross-validated F1-score?
- (b) Looking at the heatmap, is there a clear 'best region', or do several combinations perform similarly? What does that tell you about how sensitive this model is to exact hyperparameter values?
- (c) Did the cross-validated score and the final test-set score agree closely, or was there a gap? What would a large gap suggest?
- (d) Compare the tuned RBF model's test performance to the un-tuned RBF model from Part 2. Was the tuning worth the extra computation?

Part 5 — Class Imbalance and the Business Trade-off (25 minutes)

Recall from Lab 2 that churners are a minority class (roughly a quarter of customers). A model that is 'accurate' can still be nearly useless if it misses most churners. This task asks you to deliberately trade some precision for recall and justify whether that trade is worth it.

Task 5.1 — Objective

Compare your best RBF-kernel SVM with and without `class_weight='balanced'`, and reason about which is more useful for the business.

What to do

1. Using your best (C, gamma) from Part 4, train two RBF SVMs: one with `class_weight=None` (default) and one with `class_weight='balanced'`.
2. For both, report Precision, Recall, F1-score and the confusion matrix on the test set.
3. Plot either an ROC curve (`roc_curve`, `roc_auc_score`) or a Precision-Recall curve for the balanced model, and mark the operating point of the default decision threshold (0.5 equivalent).

Questions to answer

- (a) Precisely how did precision and recall move when you switched to `class_weight='balanced'`? Give the numbers.
- (b) In the context of a telecom company trying to reduce churn, is it more costly to miss an actual churner (false negative) or to wrongly flag a loyal customer as at-risk (false positive)? Justify your answer, and then say which of your two models you would recommend deploying.
- (c) SVM does not directly output a 'probability of churn' the way Logistic Regression does. How did you obtain probability-like scores for your ROC/PR curve, and what did you have to enable in `SVC()` to get them?

Part 6 — Model Showdown: SVM vs. Everything Else (20 minutes)

You now have SVM results on this exact dataset and split. Bring back your saved results from Labs 3, 4 and 6 (Decision Tree, Random Forest, Logistic Regression, KNN) and put every model on one scoreboard.

Task 6.1 — Objective

Build a single comparison table across all five algorithms and select ONE final model for production, with justification.

What to do

1. Build a comparison table with one row per model: Decision Tree (Lab 3), Random Forest (Lab 4, tuned), Logistic Regression (Lab 6), KNN (Lab 6), and your best SVM (Part 4/5). Columns: Accuracy, Precision, Recall, F1-score.
2. If you did not save exact numbers from earlier labs, briefly re-run those models here using the same train/test split as this lab, so the comparison is fair.
3. Produce ONE bar chart comparing F1-score across all five models.

Questions to answer

- (a) Which model wins on F1-score? Which wins on recall? Are they the same model?
- (b) SVM is generally slower to train than Decision Tree/Random Forest on this dataset. Given the performance differences you measured, was the extra computational cost of SVM worth it here?
- (c) Which single model would you recommend the telecom company deploy, and why? Your answer must reference at least two metrics, not just accuracy.

Part 7 — Final Predictions and Wrap-Up (15 minutes)

Package your chosen final model's predictions the way you would for a stakeholder or a Kaggle-style submission.

Task 7.1 — Objective

Generate a predictions file and write a short executive summary of your findings.

1. Using your final chosen model, generate predictions on the test set and save a CSV with (at minimum) the true label, predicted label, and predicted churn probability/score, named `svm_predictions.csv`.
2. Write a 5-8 sentence 'Executive Summary' Markdown cell: which model you chose, its key metrics, the main trade-off you made (Part 5), and one limitation of your analysis.

Final Reflection — answer briefly

- (a) What was the single most surprising result in this lab?
- (b) If you had one more hour, what would you try next (e.g. feature engineering, a different kernel, SMOTE for imbalance, a different projection for visualization)?

Time Allocation

Time	Activity
0-10 min	Part 0 — Recap and setup (reload data, reapply cleaning/encoding, split)
10-30 min	Part 1 — Scaling experiment (unscaled vs scaled linear SVM)
30-65 min	Part 2 — Kernel comparison (linear / RBF / polynomial)
65-100 min	Part 3 — 2-D PCA projection and decision-boundary visualization across C values
100-130 min	Part 4 — GridSearchCV over C and gamma, heatmap, refit and test evaluation
130-155 min	Part 5 — <code>class_weight='balanced'</code> experiment, ROC/PR curve, business trade-off
155-175 min	Part 6 — Comparison table vs. Decision Tree / Random Forest / Logistic Regression / KNN
175-180 min	Part 7 — Final predictions file and executive summary

Submission Requirements

Submit ONE completed Google Colab notebook containing all sections above, in order, with Markdown headings matching this manual's Part numbers.

At the very top of the notebook, include a Student Information block:

```
# LAB 08 — Support Vector Machines
## Student Information
Name:
Student ID:
Section:
GitHub:
Kaggle:
Dataset: Telco Customer Churn (Kaggle)
Dataset Source: <paste the Kaggle URL you used>
```

Required in the Notebook

- Every plot must be followed by a short written interpretation — a plot with no explanation earns no marks for that task.
- Every graded question above must be answered directly beneath the relevant code, in a Markdown cell.
- The comparison table in Part 6 must be visible as an actual table/DataFrame output, not just described in text.
- Push your final notebook to the GitHub repository you have been using for this course.

Grading Rubric (20 Marks)

Criterion	Marks
Part 1 — Scaling experiment: correct comparison + correct explanation	2
Part 2 — Kernel comparison: all three kernels run correctly + justified kernel choice	4
Part 3 — Margin/C visualization: correct decision-boundary plots + correct interpretation of C's effect	4
Part 4 — Hyperparameter search: correct grid search + heatmap + correct interpretation	4
Part 5 — Class imbalance: correct class_weight comparison + sound business justification	3
Part 6 — Cross-model comparison table + justified final model choice	2
Part 7 — Predictions file + executive summary + overall documentation quality	1
Total	20

Penalties

- Using `SVC()` with only default parameters and no comparison/experimentation as described above: significant deduction across Parts 2-4.
- Plots without written interpretation.
- Copied/pasted answers that do not reference your own model's actual numbers.
- Fitting the scaler (or any encoder) on the full dataset instead of the training set only — this is data leakage.
- Missing or incomplete Student Information block.